



Kandivli Education Society's
**B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE**
An Autonomous College NAAC Re-accredited 'A' Grade
ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

PROJECT REPORT
ON
EXPENSE TRACKER
IN THE PROGRAMME
MASTER IN COMPUTER APPLICATION (MCA)
SUBMITTED BY
SNEHA PANCHAL
MCA
FPMCA034A
SEMESTER II
UNDER THE GUIDANCE OF
Dr. Sadhana OJHA
ACADEMIC YEAR
2026 – 2027



Kandivli Education Society's
**B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE**

An Autonomous College

NAAC Re-accredited 'A' Grade

ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

CERTIFICATE

This is to certify that **SNEHA PANCHAL** of **SECOND** year of **MASTER IN COMPUTER APPLICATION (MCA)**, Div.: A, Roll No. **FPMCA034A** of Semester **II** (2026 - 2027) has successfully completed the Project on the topic **Expense Tracker** as per the guidelines of KES' Shroff College of Arts and Commerce, Kandivali (W), Mumbai-400067.

Teacher In-charge

Principal
Dr. Lily Bhushan

PROJECT PROPOSAL

PRN No.:

Roll no: FPMCA034A

1. Name of the Student

SNEHA PANCHAL

2. Title of the Project

EXPENSE TRACKER

3. Name of the Guide

DR. SADHANA OJHA

Signature of the Student

Signature of the Guide

Date:

Date:

Signature of the

Coordinator

Date:

.....

ABSTRACT

The Expense Tracker is a web-based application developed to simplify personal financial management by providing a secure and efficient platform to record, monitor, and analyze daily income and expenses. The system replaces traditional manual methods with a structured digital solution that improves accuracy, accessibility, and financial awareness.

The application incorporates a robust user authentication and authorization mechanism to ensure data security and privacy. Users must register and verify their identity through an OTP (One-Time Password) system, which enhances account protection and prevents unauthorized access. Role-based authorization ensures that users can only access and manage their own financial data.

Once authenticated, users can perform various operations such as adding, updating, and deleting transactions. The system categorizes income and expenses, enabling users to analyze their spending patterns effectively. A centralized dashboard provides real-time insights into total balance, income, and expenditure, helping users make informed financial decisions.

The application is built using the MERN stack, where MongoDB handles data storage, Express.js and Node.js manage backend logic, and React.js provides a dynamic and responsive user interface. The integration of OTP-based verification, secure authentication, and structured data management ensures both usability and security.

The Expense Tracker serves as a reliable solution for students and professionals to maintain financial discipline while ensuring data confidentiality and system integrity in a digital environment.

Keywords: Expense Tracker, MERN Stack, Authentication, Authorization, OTP Verification, Financial Management

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported me in the successful completion of my project, **“Expense Tracker System.”**

First and foremost, I would like to thank my project guide, **Dr.Sadhana Ojha**, for their valuable guidance, continuous support, and constructive suggestions throughout the development of this project. Their mentorship played a significant role in shaping my work.

I am also thankful to **KES Shroff College** for providing me with the necessary resources, facilities, and learning environment to carry out this project effectively.

I would like to extend my appreciation to my peers and all those who provided feedback during the testing phase. Their suggestions helped me improve the functionality and usability of the application.

Furthermore, I am deeply grateful to my family and friends for their constant encouragement, support, and understanding throughout the project development process.

Finally, I acknowledge the use of various online resources, documentation, and technical communities that helped me understand and implement technologies such as the MERN stack, authentication, and OTP-based verification.

Thank you

SNEHA PANCHAL

DECLARATION

I hereby declare that the project entitled, “**Expense Tracker System.**” done at **KES Shroff College**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **Master in Computer Application(MCA)** to be submitted as semester-II project as part of our curriculum.

SNEHA PANCHAL

Table of Contents

Chapter 1: Introduction	09
1.1 Significance of Expense Tracker System	10
1.2 Objectives of the Expense Tracker System	11
1.3 Scope of Expense Tracker System	12-13
Chapter 2: System Analysis	14
2.1 Existing System:	15
2.2 Proposed System:	16
2.3 Requirement Analysis:	17
2.4 Hardware Requirements:	18
2.5 Software Requirements:	19
2.6 Justification of Platform	20
Chapter 3: System Design	21
3.1 Module Division	22-25
3.2 Data Dictionary	26-27
3.3 ER Diagrams	28
3.4 UML Diagrams	29
3.4.1 Use Case Diagram	29-30
3.4.2 Activity Diagram	31-32
3.4.3 Class Diagram	33-34
3.4.4 Sequence Diagram	35-36
3.4.5 Data Flow Diagram	37-39
Chapter 4: Implementation and Testing	40
4.1 Implementation Overview	41-42
4.2 Module wise Implementation	43-45

4.3 Database Implementation.....	46
4.4 Security Implemenatation.....	47
4.5 Testing Strategy.....	48
4.6 Test Cases.....	49-50
Chapter 5: Results and Discussions.....	51
5.1 Results.....	52-55
5.2 Functionality Evaluation:	56
5.3 User Experience Assessment:	57
5.4 Data Security and Integrity:	58
Chapter 6: Conclusion and Future Work	59
6.1 Conclusion.....	60
6.2 Future Scope.....	61
6.3 Limitations.....	62
Chapter 7: References.....	63
7.1 References.....	64

Chapter 1: Introduction

1.1 Significance of Expense Tracker System

The Expense Tracker System plays an important role in helping individuals manage their daily financial activities in a structured and efficient manner. In today's digital age, many people still rely on manual methods or basic tools to track their income and expenses, which often leads to poor financial planning, calculation errors, and lack of clarity in spending habits.

The significance of this application lies in its ability to provide a simple and centralized platform where users can record, update, and monitor their financial transactions in real time. Unlike traditional methods, the system ensures better organization of financial data and reduces the chances of errors and data loss.

The application also incorporates secure authentication mechanisms, including OTP-based verification and authorization, which ensures that user data remains protected and accessible only to authorized individuals. This adds a layer of security and reliability to the system.

Additionally, the system provides a dashboard that gives users a clear overview of their total income, expenses, and remaining balance. This helps users understand their spending patterns and make informed financial decisions.

The use of modern web technologies such as the MERN stack enhances the performance, scalability, and responsiveness of the application. Its simplicity, accessibility, and efficiency make it a practical solution for students and working professionals who want to improve their financial discipline and budgeting habits.

1.2 Objectives of the Expense Tracker System

The primary objective of the Expense Tracker System is to provide a simple, secure, and efficient web-based solution for managing personal financial transactions. The specific objectives of the system are as follows:

1. To replace manual expense tracking methods with a structured digital system.
2. To enable users to record, update, and delete income and expense transactions easily.
3. To provide categorized tracking of financial activities for better understanding of spending patterns.
4. To display real-time financial information such as total balance, income, and expenses through a dashboard.
5. To implement secure user authentication using login credentials and OTP-based verification.
6. To ensure proper authorization so that users can only access and manage their own data.
7. To store and manage financial data efficiently using a database system.
8. To develop a responsive and user-friendly interface using modern web technologies.

1.3 Scope of Expense Tracker System

The scope of the Expense Tracker System defines the functionalities and services provided by the application to support personal financial management. The system is designed to offer a simple and efficient way for users to track and manage their daily income and expenses. The scope includes the following components:

1. User Management

Secure user registration and login functionality

- OTP-based verification for enhanced security
- User authentication and authorization to protect personal data
- Basic profile management features

2. Transaction Management

- Recording of daily income and expense transactions
- Ability to add, update, and delete transactions
- Categorization of transactions for better organization
- Storage of transaction history

3. Financial Tracking

- Real-time calculation of total balance
- Monitoring of income and expenses
- Overview of financial status through dashboard

4. Dashboard and Visualization

- Display of summarized financial data
- Simple visual representation of income and expenses (if implemented)
- Easy understanding of spending patterns

5. Data Management

- Storage of user and transaction data in database
- Efficient retrieval and updating of records
- Structured data handling using backend APIs

6. Security Features

- Implementation of authentication mechanisms (login system)

- OTP-based verification for user identity
- Authorization to restrict access to user-specific data

7. Web Accessibility

- Web-based application accessible through browser
- Responsive user interface for better user experience
- Easy navigation for non-technical users

Chapter 2: System Analysis

2.1 Existing System

In the existing system, most individuals manage their personal finances using traditional methods such as manual record-keeping, notebooks, or basic spreadsheet tools like Excel. These methods are simple to use but are often inefficient, time-consuming, and prone to human errors such as incorrect calculations or missing entries.

Many users rely on memory to track daily expenses, which leads to poor financial awareness and unplanned spending. Manual systems also lack proper organization, making it difficult to analyze past transactions or identify spending patterns.

There are several existing digital solutions, such as basic expense tracking applications and mobile apps, that allow users to record income and expenses. While these applications provide some level of convenience, they often have limitations such as:

- Limited customization and flexibility
- Lack of secure authentication mechanisms
- No OTP-based verification for user identity
- Restricted data control and privacy concerns in some cases

Additionally, many basic systems do not provide a structured dashboard or real-time financial overview, which makes it difficult for users to get a clear understanding of their financial status.

As a result, users still face challenges in maintaining accurate records, ensuring data security, and effectively analyzing their financial activities. These limitations highlight the need for a more secure, organized, and user-friendly expense tracking system.

2.2 Proposed System

The proposed system, **Expense Tracker System**, is a web-based application designed to overcome the limitations of traditional manual methods and basic expense tracking tools. The system focuses on providing a secure, organized, and user-friendly platform for managing personal financial transactions efficiently.

The application introduces a structured digital approach where users can record, update, and manage their income and expenses in real time. All financial data is stored in a centralized database, ensuring easy access, better organization, and reduced chances of data loss.

A key feature of the proposed system is the implementation of secure authentication mechanisms, including login credentials and OTP (One-Time Password) verification. This ensures that only authorized users can access their accounts, thereby improving data security and privacy.

The system also provides a dashboard that offers a clear overview of total balance, income, and expenses. This helps users understand their financial position instantly and make better financial decisions. Transactions can be categorized, allowing users to analyze their spending patterns more effectively.

Unlike traditional methods, the system automates calculations such as total income, total expenses, and remaining balance, reducing manual effort and errors. The use of the MERN stack ensures fast performance, scalability, and a responsive user interface.

The proposed system eliminates inefficiencies of manual tracking, enhances data security through authentication and authorization, and provides a simple yet effective solution for personal financial management.

Key Feature of the Proposed System

“The key strength of the Expense Tracker System lies in its secure and user-friendly digital platform that combines real-time expense tracking with OTP-based authentication, ensuring both financial awareness and data protection.”

2.3 Requirement Analysis

Requirement analysis defines the functional and non-functional requirements of the **Expense Tracker System**, ensuring that the application meets user needs effectively.

Functional Requirements

- User registration and secure login functionality
- OTP (One-Time Password) based verification for user authentication
- Authorization to ensure users can access only their own data
- Recording of income and expense transactions
- Ability to add, update, and delete transactions
- Categorization of transactions for better organization
- Automatic calculation of total balance, income, and expenses
- Display of transaction history
- Dashboard to provide real-time financial overview

Non-Functional Requirements

- Secure data storage and protection of user information
- Reliable authentication and authorization mechanisms
- Fast response time for user actions
- User-friendly and responsive interface
- Scalability to handle increasing number of users and transactions
- Availability of the system for continuous access
- Efficient database performance and data retrieval

2.4 Hardware Requirements

The hardware requirements for the **Expense Tracker System** are categorized based on development and end-user usage.

Development Hardware

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended for smooth development)
- **Storage:** At least 256 GB SSD
- **System Type:** 64-bit operating system

Server Hardware

- **Processor:** Dual-core or higher
- **RAM:** Minimum 4 GB
- **Storage:** 50 GB or more
- **Internet Connectivity:** Required for API testing and package installation

End-User Hardware

- **Device:** Desktop, Laptop, or Smartphone
- **RAM:** Minimum 4 GB
- **Storage:** At least 100 MB free space
- **Browser:** Any modern web browser (Chrome, Edge, etc.)
- **Internet Connectivity:** Required to access the web application

2.5 Software Requirements

The software requirements for the **Expense Tracker System** include the technologies and tools used for development, execution, and testing of the application.

Frontend

- **React.js** – for building the user interface
- **HTML, CSS, JavaScript** – for designing and structuring the web pages
- **Visual Studio Code** – for development and coding

Backend

- **Node.js** – runtime environment for server-side execution
- **Express.js** – framework for building RESTful APIs
- **JWT (JSON Web Token)** – for authentication and authorization
- **OTP Verification System** – for secure user validation

Database

- **MongoDB** – NoSQL database for storing user and transaction data
- **MongoDB Compass** – for database visualization and management

Tools & Technologies

- RESTful APIs for communication between frontend and backend
- Git and GitHub for version control
- Postman for API testing
- npm (Node Package Manager) for managing dependencies

Operating Systems

- **Development:** Windows / Linux / macOS
- **Client:** Any operating system with a modern web browser
- **Server:** Node.js supported environment (local or cloud)

2.6 Justification of Platform

The Expense Tracker System is developed as a web-based application using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. This platform is chosen due to its flexibility, performance, and suitability for modern web application development.

A web-based platform is preferred over mobile-only solutions because it allows users to access the system from any device with a web browser, including desktops, laptops, and smartphones. This increases accessibility without requiring platform-specific installations.

React.js is used for frontend development due to its component-based architecture, which enables the creation of a dynamic and responsive user interface. It improves user experience by allowing fast rendering and smooth interaction.

Node.js and Express.js are used for backend development because they support fast and scalable server-side operations. They are well-suited for handling API requests and managing application logic efficiently.

MongoDB is selected as the database due to its flexible schema design and ability to handle large volumes of data. It allows efficient storage and retrieval of user and transaction records.

Additionally, the platform supports secure authentication mechanisms such as JWT-based login and OTP (One-Time Password) verification, ensuring user data protection and system reliability.

The overall MERN stack provides a cost-effective, scalable, and efficient solution for building the Expense Tracker System, making it suitable for personal financial management applications.

Chapter 3: System Design

3.1 Module Division

The **Expense Tracker System** is designed using a modular architecture to ensure better maintainability, scalability, and clarity. Each module is responsible for a specific functionality and interacts with other modules through well-defined APIs. This modular structure simplifies development, debugging, and future enhancements.

3.1.1 User Management Module

This module handles all user-related operations and ensures secure access to the system.

Functions:

- User registration
- Secure login using credentials
- OTP (One-Time Password) verification
- Authentication using JWT
- Basic profile management (name, email)

Related Entity:

- User

Purpose:

To ensure secure user authentication, identity verification, and controlled access to the system.

3.1.2 Transaction Management Module

This module manages all financial transactions recorded by the user.

Functions:

- Add income and expense transactions
- Update existing transactions
- Delete transactions
- Categorize transactions (e.g., food, travel, etc.)
- Maintain transaction history

Related Entity:

- Transaction

Purpose:

To provide a structured way to record and manage financial activities efficiently.

3.1.3 Dashboard Module

This module provides a summarized view of the user's financial data.

Functions:

- Display total balance
- Show total income and expenses
- Provide quick overview of financial status

Purpose:

To help users quickly understand their financial position and make informed decisions.

3.1.4 Authentication & Authorization Module

This module ensures system security and data protection.

Functions:

- OTP-based user verification
- JWT token generation and validation
- Protecting routes from unauthorized access
- Session handling

Purpose:

To ensure that only authenticated users can access and manage their data securely.

3.1.5 Database Management Module

This module handles data storage and retrieval.

Functions:

- Store user details securely
- Store transaction records
- Fetch and update data efficiently
- Maintain data consistency

Related Entity:

- **User**
- **Transaction**

Purpose:

To manage all application data in a structured and reliable manner using a database.

3.1.6 Category Management Module

This module handles the organization of transactions into different categories.

Functions:

- Create and manage categories (e.g., Food, Travel, Bills)
- Assign categories to income and expense transactions
- Filter transactions based on category

Related Entity:

- **Category**

Purpose:

To help users organize their financial data and better understand spending patterns.

3.1.7 Security Module

This module ensures data protection and secure system access.

Functions:

- User authentication using login credentials
- OTP-based verification for user identity
- JWT-based authorization for protected routes
- Secure API access

Purpose:

To protect sensitive user data and prevent unauthorized access to the system.

3.1.8 Reporting and Financial Summary Module

This module provides insights into the user's financial activities.

Functions:

- Display total income and expenses
- Show remaining balance

- Provide basic summaries of transactions
- Category-wise analysis

Purpose:

To help users understand their financial status and make better financial decisions.

3.2 Data Dictionary

The Data Dictionary defines the structure of data used in the **Expense Tracker System**. It includes details of collections (tables), field names, data types, and their purpose to ensure consistency and clarity in database design.

3.2.1 User Collection (users)

Field Name	Data Type	Description
_id	ObjectId (PK)	Unique identifier for user
name	String	Name of the user
email	String (Unique)	Registered email address
password	String	Encrypted password
otp	String	One-Time Password for verification
otp_expiry	Date	OTP expiration time
is_verified	Boolean	Email/OTP verification status
createdAt	Date	Account creation date
updatedAt	Date	Last update time

3.2.2 Transaction Collection (transactions)

Stores all income and expense records.

Field Name	Data Type	Description
_id	ObjectId (PK)	Unique transaction ID
user_id	ObjectId (FK)	Reference to user
type	String	Income / Expense
amount	Number	Transaction amount
category	String	Type of expense (Food, Travel, etc.)
description	String	Transaction details

date	Date	Date of transaction
createdAt	Date	Entry creation time
updatedAt	Date	Last update time

3.2.2 OTP Verification Structure

OTP-related fields are used for secure user verification.

Field Name	Data Type	Description
otp	String	One-Time Password
otp_expiry	Date	Expiration time of OTP
is_verified	Boolean	Verification status

3.2.3 Dashboard Data (Derived Data)

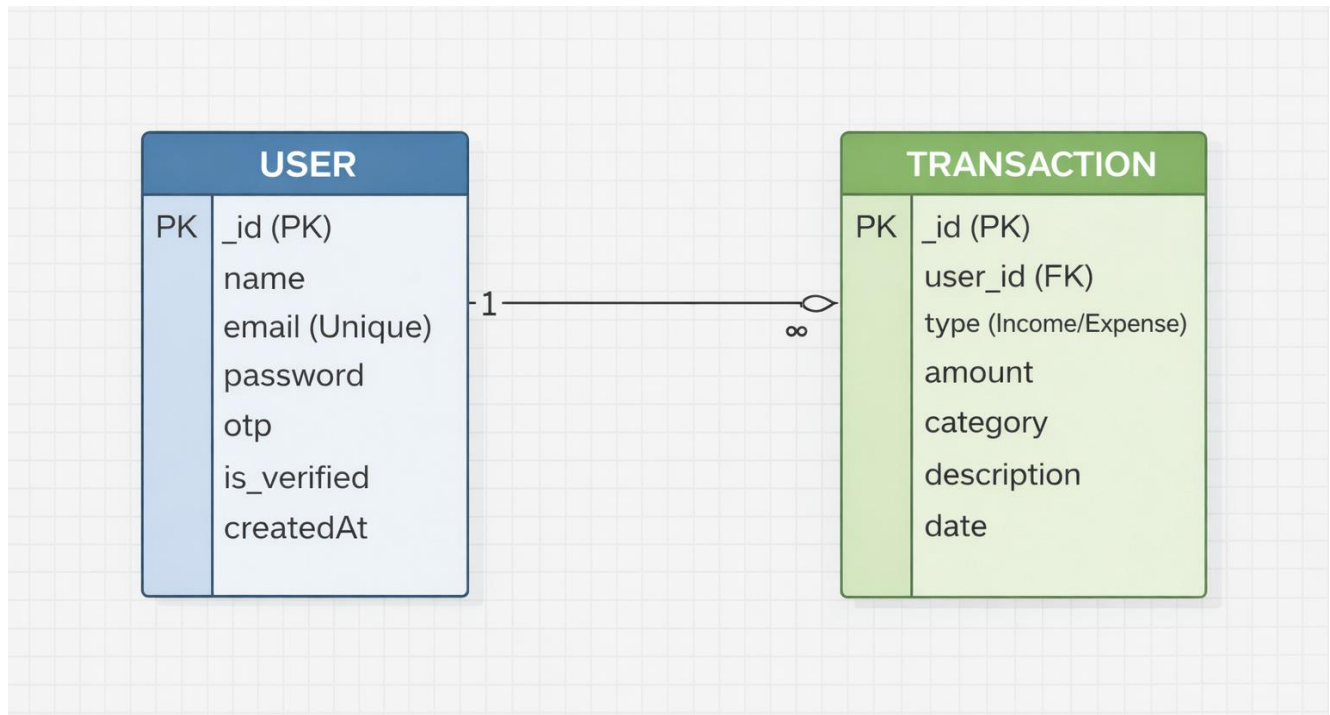
This is not a separate collection but calculated dynamically from transactions.

Data Element	Description	Data Element
Total Income	Sum of all income transactions	Total Income
Total Expense	Sum of all expense transactions	Total Expense
Balance	Income – Expense	Balance

3.3 ER Diagrams

An Entity Relationship (ER) Diagram represents the logical structure of the **Expense Tracker System** database. It identifies the main entities, their attributes, and the relationships between them, ensuring proper data organization and integrity.

The ER diagram of the Expense Tracker System primarily focuses on user data and financial transactions. The system is designed as a single-user-based application where each user manages their own set of income and expense records.



3.4 UML Diagrams

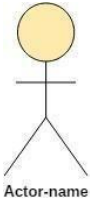
Unified Modeling Language (UML) diagrams are used to represent the structure and behavior of the **Expense Tracker System**. These diagrams help in understanding how users interact with the system and how different components work together. UML provides a clear visual representation of system functionalities and workflows.

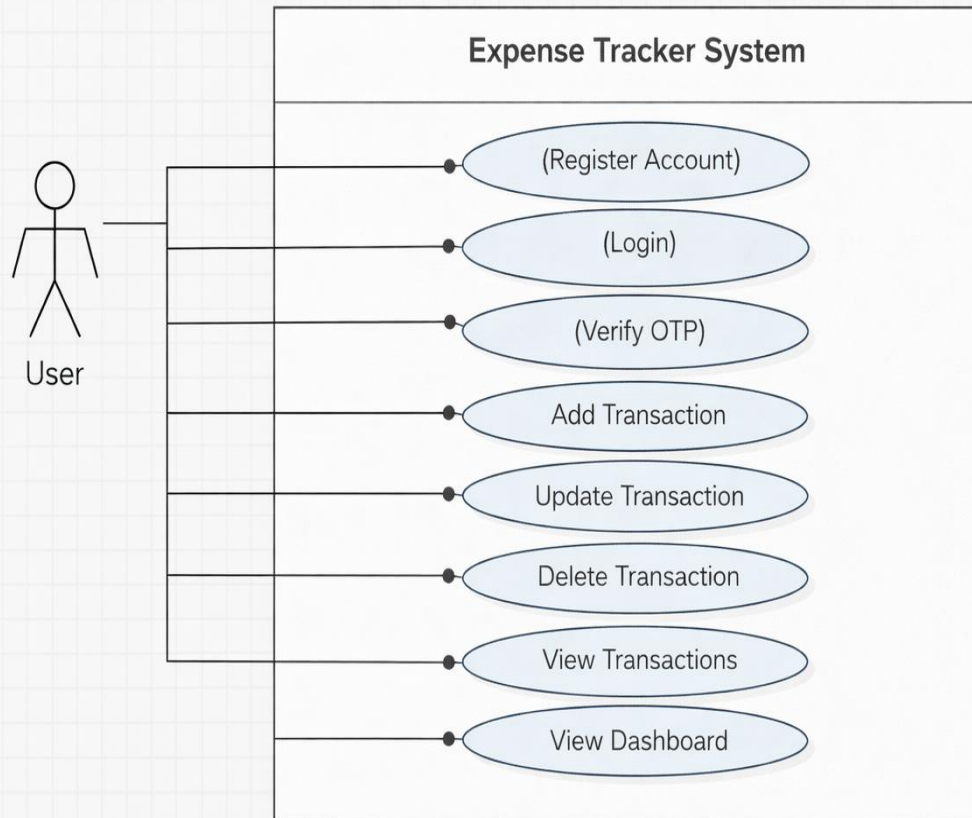
The following UML diagrams are used in this project:

3.4.1 Use case Diagram

A Use Case Diagram represents the interaction between the user (actor) and the system. It shows the functionalities provided by the system and how the user interacts with them.

In the Expense Tracker System, the primary actor is the **User**, who interacts with the system to manage financial data securely.

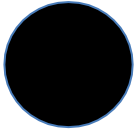
Symbol	Name	Description
	Use Case	Represents a system functionality
	Actor	Represents the user interacting with the system
	Association	Shows interaction between actor and system

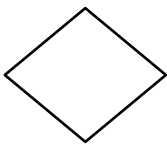


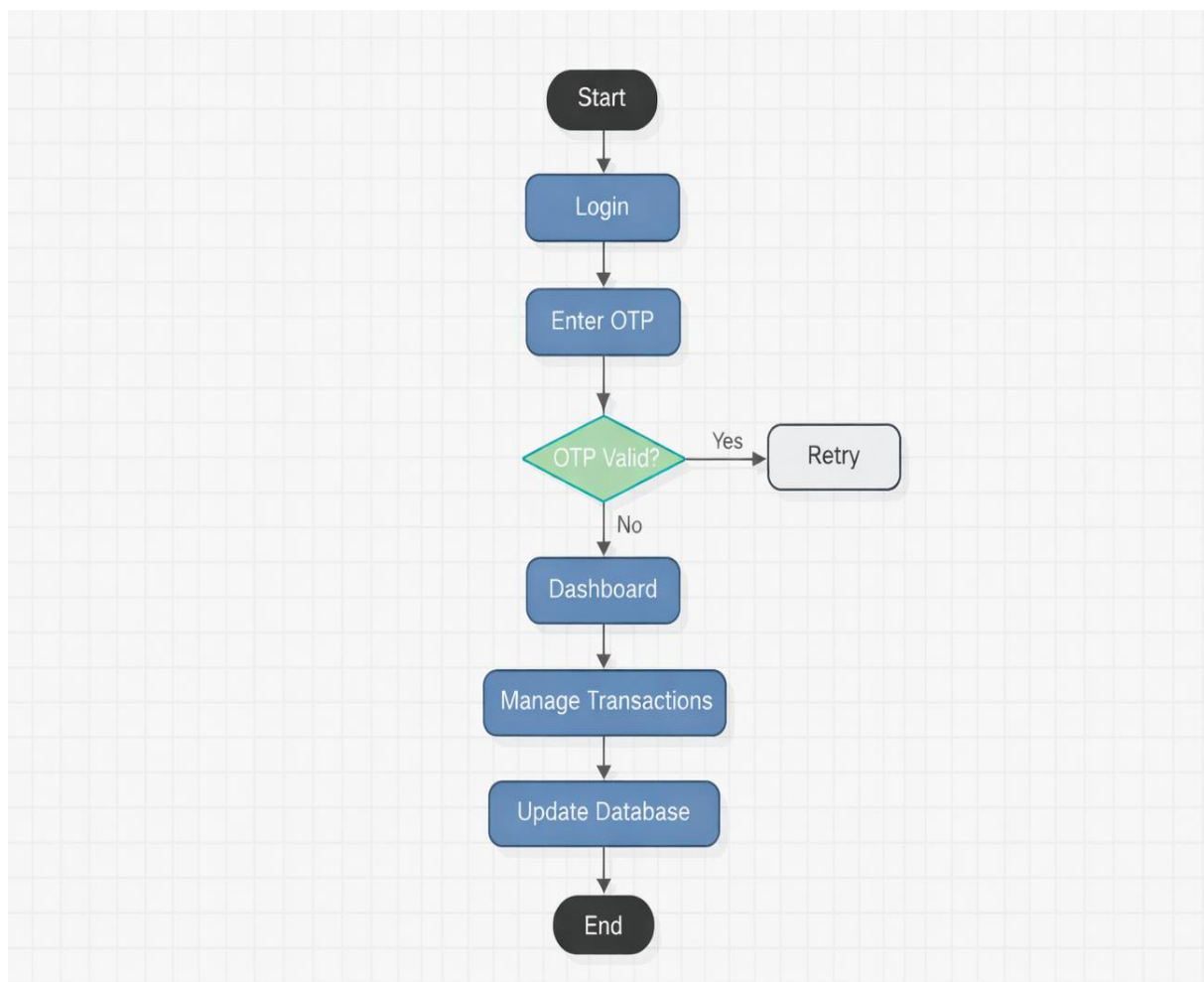
3.4.2 Activity Diagram

An Activity Diagram is a behavioral UML diagram that represents the flow of activities within the **Expense Tracker System**. It shows how the user interacts with the system step-by-step, from authentication to managing transactions.

The diagram helps in understanding the sequence of operations, decision points, and overall workflow of the system.

Symbol	Name	Description
	Start	Represents the beginning of the process
	End	Represents the termination of the process
	Activity	Represents an operation or task
	Flow Line	Shows direction of control

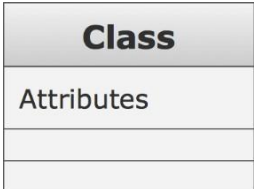
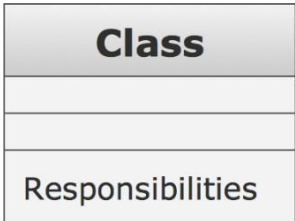
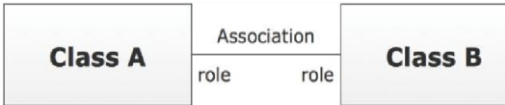
	Decision	Represents a condition (Yes/No)
---	----------	---------------------------------

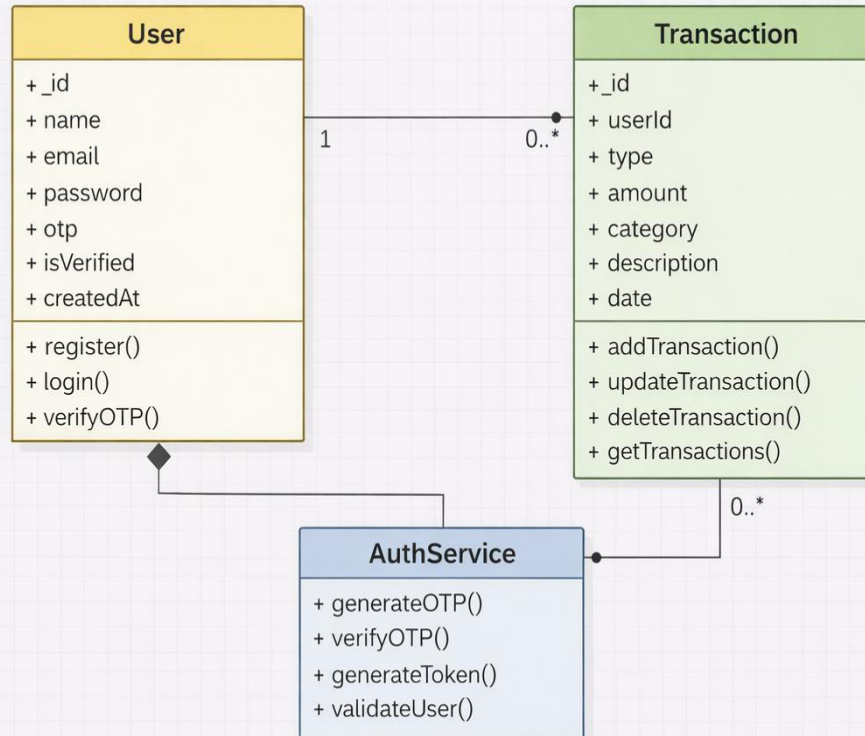


3.4.3 Class Diagram

A UML Class Diagram represents the static structure of the **Expense Tracker System**. It shows the main classes, their attributes (data members), methods (functions), and relationships between them.

The system is designed using core classes such as **User**, **Transaction**, and **AuthService**, which handle user data, financial records, and authentication processes.

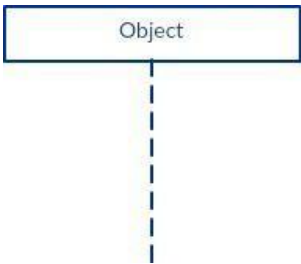
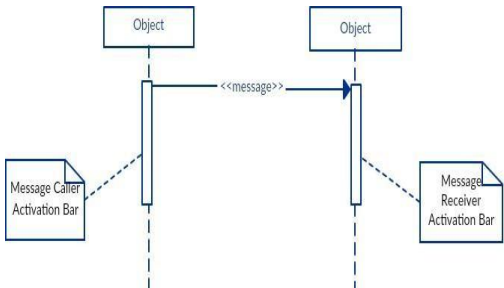
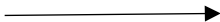
Symbol	Name	Description
	Class	Represents a class
	Attribute	Variables/data members
	Responsibility	Functions/operations.
	Association	Relationship between classe0073

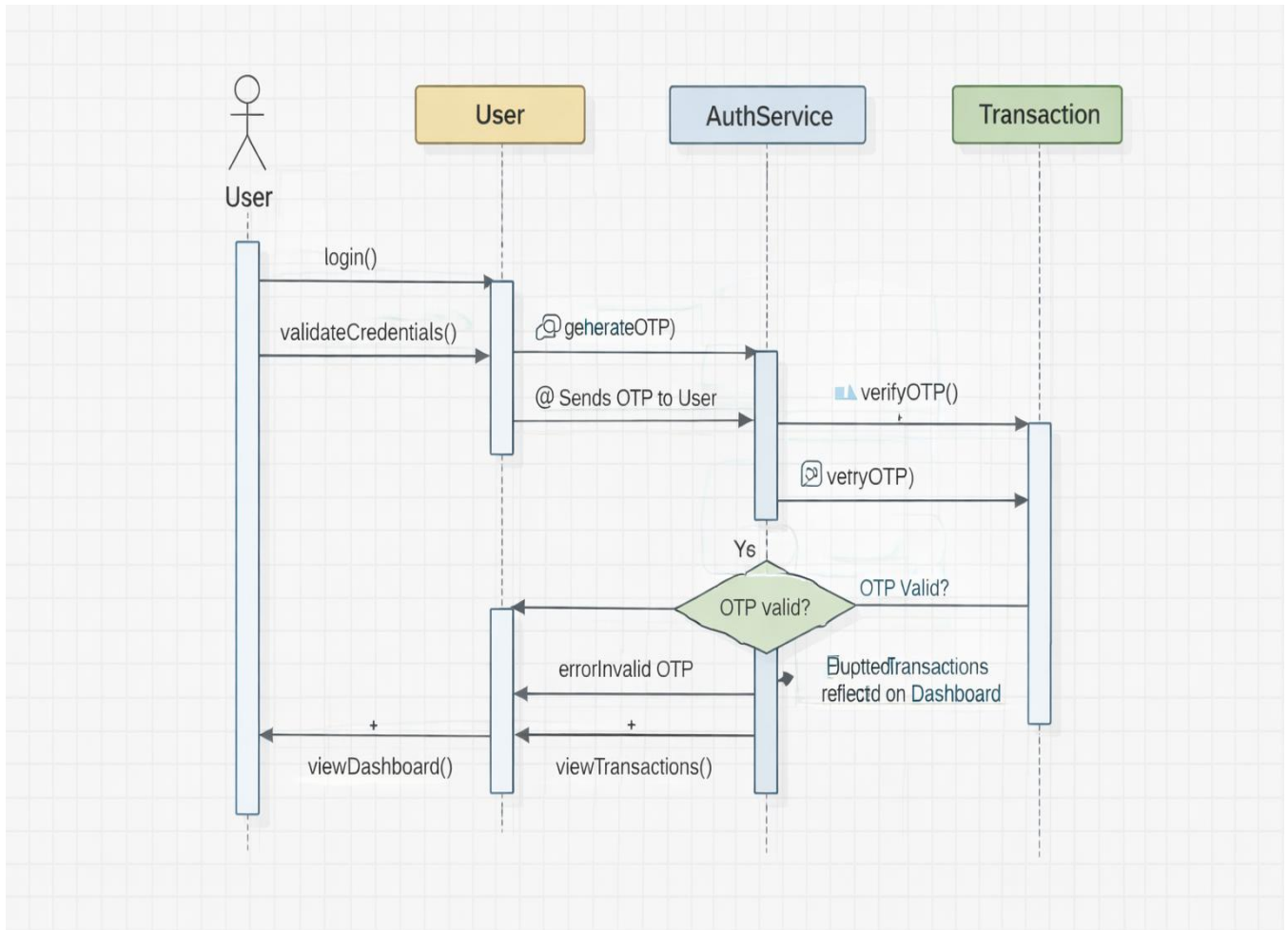


3.4.4 Sequence Diagram

A Sequence Diagram is a UML interaction diagram that represents the sequence of messages exchanged between different components of the **Expense Tracker System** over time. It shows how the user interacts with the system and how different modules such as authentication and transaction management communicate with each other.

The sequence diagram focuses on the chronological flow of operations, especially during user authentication and transaction handling.

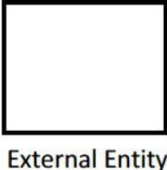
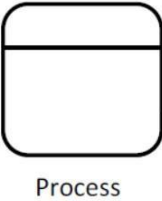
Symbol	Name	Description
	Lifeline Notation	A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram. No two lifeline notations should overlap each other.
	Activation Bars	Activation bar is the box placed on the lifeline. It is used to indicate that an object is active (or instantiated) during an interaction between two objects. The length of the rectangle indicates the duration of the objects staying active. In a sequence diagram, an interaction between two objects occurs when one object sends a message to another.
	Synchronous Message	A synchronous message is used when the sender waits for the receiver to process the message and return before carrying on with another message. The arrowhead used to indicate this type of message is a solid one.



3.4.5 DFD Diagram

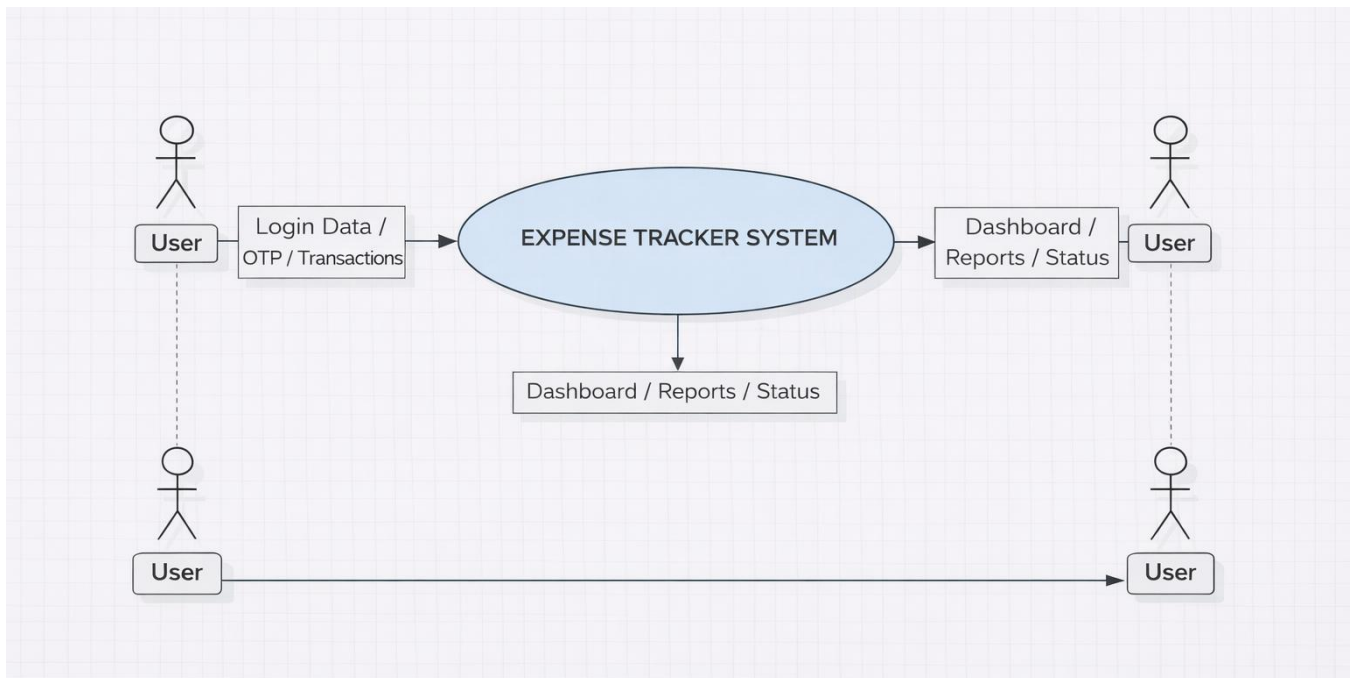
A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through the **Expense Tracker System**. It illustrates the flow of information between the user, system processes, and the database.

The DFD focuses on how input data is processed, stored, and returned as output, without detailing internal logic.

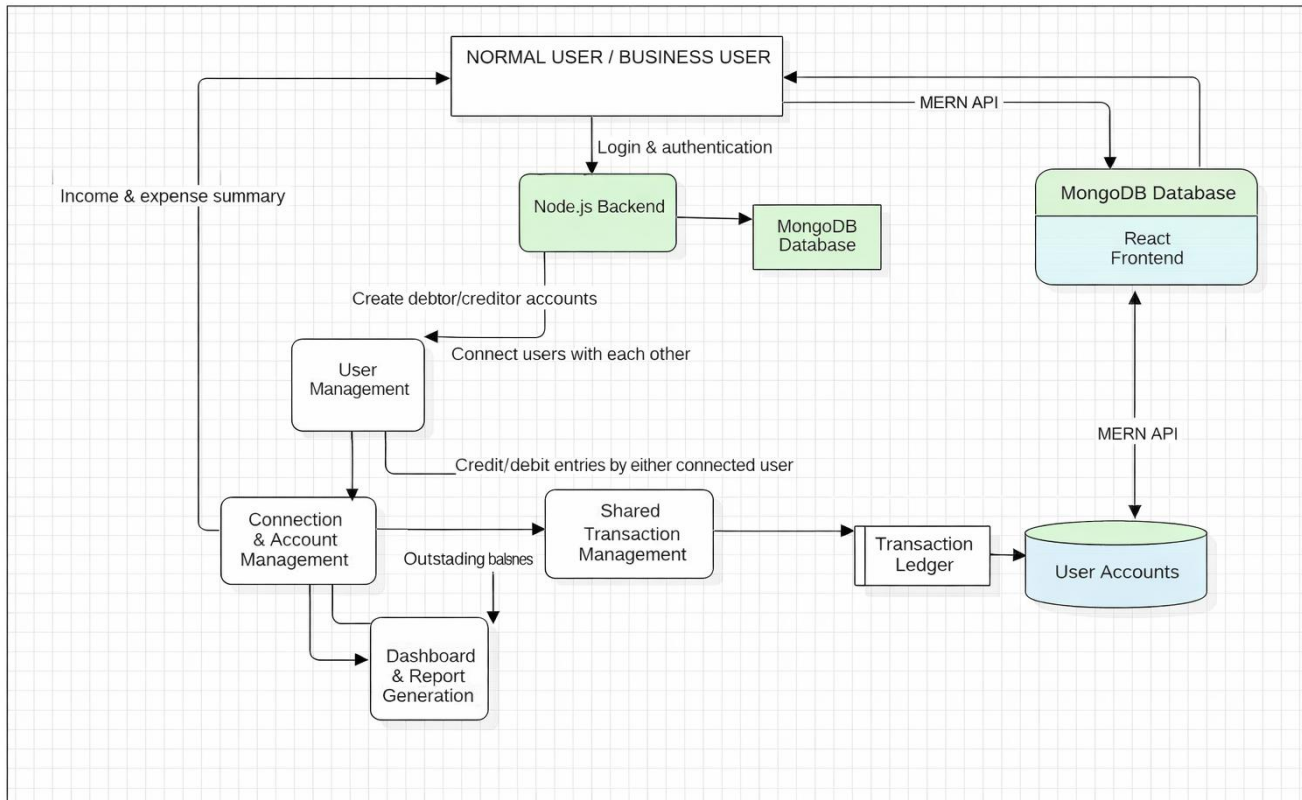
Symbol	Name	Description
	External entity	External entities are objects outside the system with which system communicates. These are sources and destinations of the system inputs and outputs.
	Process	A process receives input data and process output data with a different form or content. Every process has a name that identifies the function it performs.
	Data flow	Data flow is the path for data to move from one part of the system to another. It may be a single data element or set of data element. The symbol of data flow is the arrow. The arrow shows the flow direction.

 Data store	Data Stores	Represented by open-ended rectangles or parallel lines, these are repositories where data is stored within the system. Data stores can be databases, files, or any other storage mechanism.
---	--------------------	---

Context Diagram (Level 0 DFD)



Level-1 DFD



Chapter 4: Implementation and Testing

4.1 Implementation Overview

The **Expense Tracker System** is implemented as a full-stack web application using the **MERN stack** architecture. The system is designed to provide a simple, responsive, and secure platform for managing personal expenses.

The implementation follows a **modular and layered architecture**, ensuring scalability, maintainability, and separation of concerns between frontend, backend, and database layers.

Core Implementation Focus

The system is developed to handle the following functionalities:

- **Expense Management**
Users can add, update, delete, and view daily expenses with categories and timestamps.
 - **User Authentication & Security**
Secure login and registration using token-based authentication (JWT).
 - **Real-Time Data Handling**
Dynamic updates of expenses without reloading the page.
 - **Data Visualization**
Graphical representation of spending patterns using charts.
 - **CRUD Operations**
Efficient handling of Create, Read, Update, Delete operations on expense data.
-

Technology Stack Used

- **Frontend:** React.js
Used for building an interactive and responsive user interface.
 - **Backend:** Node.js with Express.js
Handles API requests, business logic, and server-side processing.
 - **Database:** MongoDB
Stores user and expense data in a flexible JSON-like format.
 - **API Communication:** RESTful APIs
Enables communication between frontend and backend.
-

System Architecture

The application follows a **three-tier architecture**:

1. **Presentation Layer (Frontend)**
Developed using React, responsible for UI rendering and user interaction.
 2. **Application Layer (Backend)**
Handles request processing, authentication, and business logic.
 3. **Data Layer (Database)**
Stores all persistent data such as user credentials and expense records.
-

Data Management

- Data is stored in collections such as:

- Users
- Expenses
- Each expense record includes:
 - Amount
 - Category
 - Date
 - Description

MongoDB ensures **flexibility and scalability** compared to rigid relational schemas.

4.2 Module Wise Implementation

4.2.1 User Authentication Module

Implemented Features:

- User registration and login
- Password hashing for security
- Token-based authentication using JWT
- Session management (login/logout)

Key Components:

- User Model (MongoDB Schema)
 - Authentication Controller
 - Middleware for protected routes
-

4.2.2 Expense Management Module

This is the core module of your system — if this is weak, your whole project is weak.

Implemented Features:

- Add new expense entries
- Edit and delete existing expenses
- Categorization (Food, Travel, Bills, etc.)
- Date-wise expense tracking
- Description and amount fields

Key Components:

- Expense Model
 - Expense Controller
 - API Routes for CRUD operations
-

4.2.3 Dashboard & Analytics Module

Implemented Features:

- Display total expenses
- Category-wise expense summary
- Monthly/weekly expense overview
- Visual representation using charts

Key Components:

- Dashboard Component (React)
 - Chart integration (e.g., Chart.js)
 - Aggregation logic in backend
-

4.2.4 Data Management Module

Implemented Features:

- Storage of user and expense data in MongoDB
- Efficient retrieval using queries
- Update and deletion handling
- Data validation before storage

Key Components:

- Database Collections (Users, Expenses)
 - Mongoose Schemas
 - Query Handling via Express.js APIs
-

4.2.5 UI/Frontend Module

Implemented Features:

- Responsive user interface
- Form handling for adding expenses

- Dynamic data rendering
- Navigation between pages (Login, Dashboard, History)

Key Components:

- React Components
- State Management (useState / useEffect)
- API Integration with Node.js backend

4.3 Database Implementation

The database for the Expense Tracker System is implemented using MongoDB, which is a NoSQL database designed for flexible and scalable data storage.

Unlike relational databases, MongoDB uses a document-based structure, where data is stored in JSON-like format (BSON). This eliminates the need for complex joins and rigid schemas.

Collections Used

- Users Collection
 - Stores user information such as:
 - Name
 - Email
 - Password (hashed)
- Expenses Collection
 - Stores all expense records:
 - Amount
 - Category
 - Date
 - Description
 - User ID (reference)

Data Handling

- Data is managed using Mongoose (ODM for MongoDB)
- Each document is uniquely identified using ObjectId
- Relationships are handled through referencing (User → Expenses)
- CRUD operations are performed via backend APIs

Key Features

- Schema-based validation using Mongoose
- Automatic timestamping (createdAt, updatedAt)
- Efficient querying and filtering
- Scalable structure for future enhancements

4.4 Security Implementation

Security in the Expense Tracker System is implemented to protect user data and prevent unauthorized access.

Security Mechanisms Used

- **Authentication**
Implemented using JWT (JSON Web Token) for secure login sessions
- **Password Security**
Passwords are encrypted using hashing (e.g., bcrypt)
- **Authorization**
Protected routes ensure only authenticated users can access their data
- **Secure API Design**
Backend APIs built using Express.js restrict unauthorized requests

Data Protection Measures

- User-specific data isolation (one user cannot access another's data)
- Input validation to prevent invalid or malicious data
- Basic error handling to avoid sensitive data exposure

4.5 Testing Strategy

Software testing ensures that the Expense Tracker System works correctly, securely, and reliably under different conditions.

Since your project is small-scale, don't overact with "enterprise-level testing" — keep it realistic but structured.

Testing Approach Used

The system follows a **basic Agile-inspired testing approach**, where testing is performed alongside development.

Testing Levels

1. Unit Testing

- Individual backend functions are tested
- Example: Adding an expense, deleting a record

2. API Testing

- APIs are tested using tools like Postman
- Validates request-response cycle

3. Integration Testing

- Ensures frontend and backend work together properly
- Example: React form → API → Database → Response

4. System Testing

- Entire application is tested as a complete system
- Checks overall workflow and performance

5. User Acceptance Testing (UAT)

- Tested from user perspective
- Ensures the system meets basic user needs

4.6 Test Cases

Test Case Table – Expense Tracker System

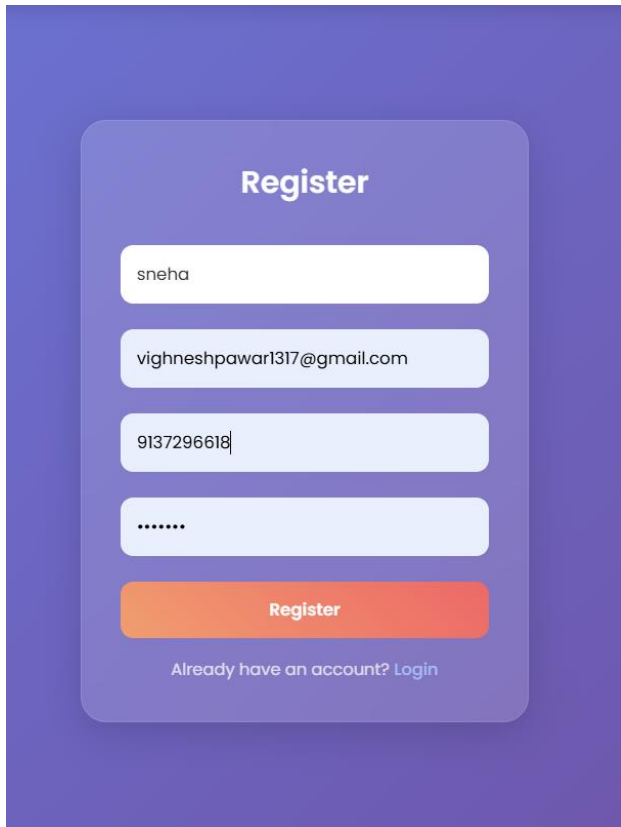
TC ID	Test Case Name	Description	Input Data	Expected Output	Result
01	User Registration	Verify new user can register	Valid name, email, password	User registered successfully	Pass
02	User Login	Verify login functionality	Valid email & password	Login successful, token generated	Pass
03	Invalid Login	Check login with wrong credentials	Wrong password	Error message displayed	Pass
04	Add Expense	Verify adding new expense	Amount ₹500, Category Food	Expense added successfully	Pass
05	View Expenses	Fetch all user expenses	Valid user token	List of expenses displayed	Pass
06	Update Expense	Modify existing expense	Change amount ₹500 → ₹700	Expense updated successfully	Pass
07	Delete Expense	Remove expense record	Valid expense ID	Expense deleted successfully	Pass
08	Category Filtering	Filter expenses by category	Category = Travel	Only travel expenses shown	Pass
09	Date-wise Filtering	Filter by date	Select date range	Relevant expenses displayed	Pass
10	Dashboard Summary	Check total expense calculation	Multiple entries	Correct total displayed	Pass
11	Data Persistence	Verify data storage	Restart system	Data remains unchanged	Pass
12	Unauthorized Access	Access without login	No token	Access denied	Pass

13	Logout	Verify logout functionality	Logout request	Session terminated	Pass
14	API Response Test	Validate backend API	API call via Postman	Correct JSON response	Pass

Chapter 5: Results and Discussions

5.1 Results

Register



The Register form is displayed on a purple gradient background. It features a white rounded rectangle containing the title "Register" in bold. Below the title are four input fields: a text field with "sneha", an email field with "vighneshpawar1317@gmail.com", a phone number field with "9137296618", and a password field with "*****". A red "Register" button is positioned below the inputs. At the bottom, a link "Already have an account? Login" is shown in blue.

Register

sneha

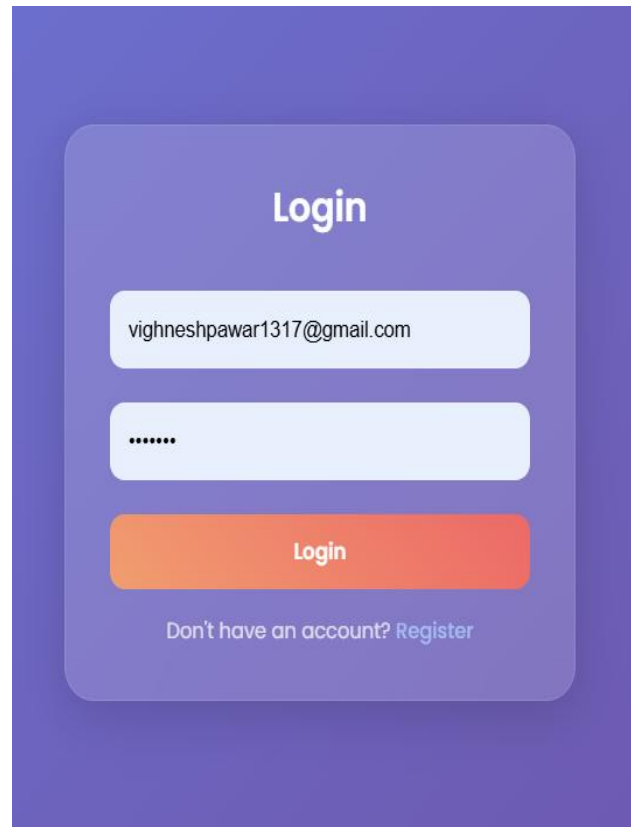
vighneshpawar1317@gmail.com

9137296618

Register

[Already have an account? Login](#)

Login



The Login form is displayed on a purple gradient background. It features a white rounded rectangle containing the title "Login" in bold. Below the title are two input fields: an email field with "vighneshpawar1317@gmail.com" and a password field with "*****". A red "Login" button is positioned below the inputs. At the bottom, a link "Don't have an account? Register" is shown in blue.

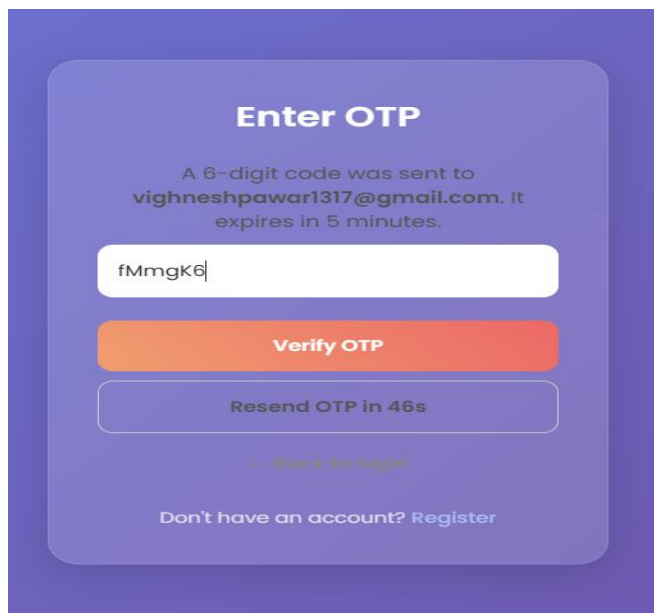
Login

vighneshpawar1317@gmail.com

Login

[Don't have an account? Register](#)

OTP



The Enter OTP form is displayed on a purple gradient background. It features a white rounded rectangle containing the title "Enter OTP" in bold. Below the title is a message: "A 6-digit code was sent to vighneshpawar1317@gmail.com. It expires in 5 minutes." Below the message is an input field with "fMmgK6". A red "Verify OTP" button is positioned below the input field. Below the button is a button "Resend OTP in 46s" with a green timer. At the bottom, a link "Don't have an account? Register" is shown in blue.

Enter OTP

A 6-digit code was sent to vighneshpawar1317@gmail.com. It expires in 5 minutes.

fMmgK6

Verify OTP

Resend OTP in 46s

[Don't have an account? Register](#)

Add Expense

 Expense Tracker

DashboardMy PortfolioNewsLogout

Total Expenses
₹30000

Add Expense

xghdjhhfj

300000

chjhjhjg

12-May-2026

hgfu jkgkugkhujk

Add Expense

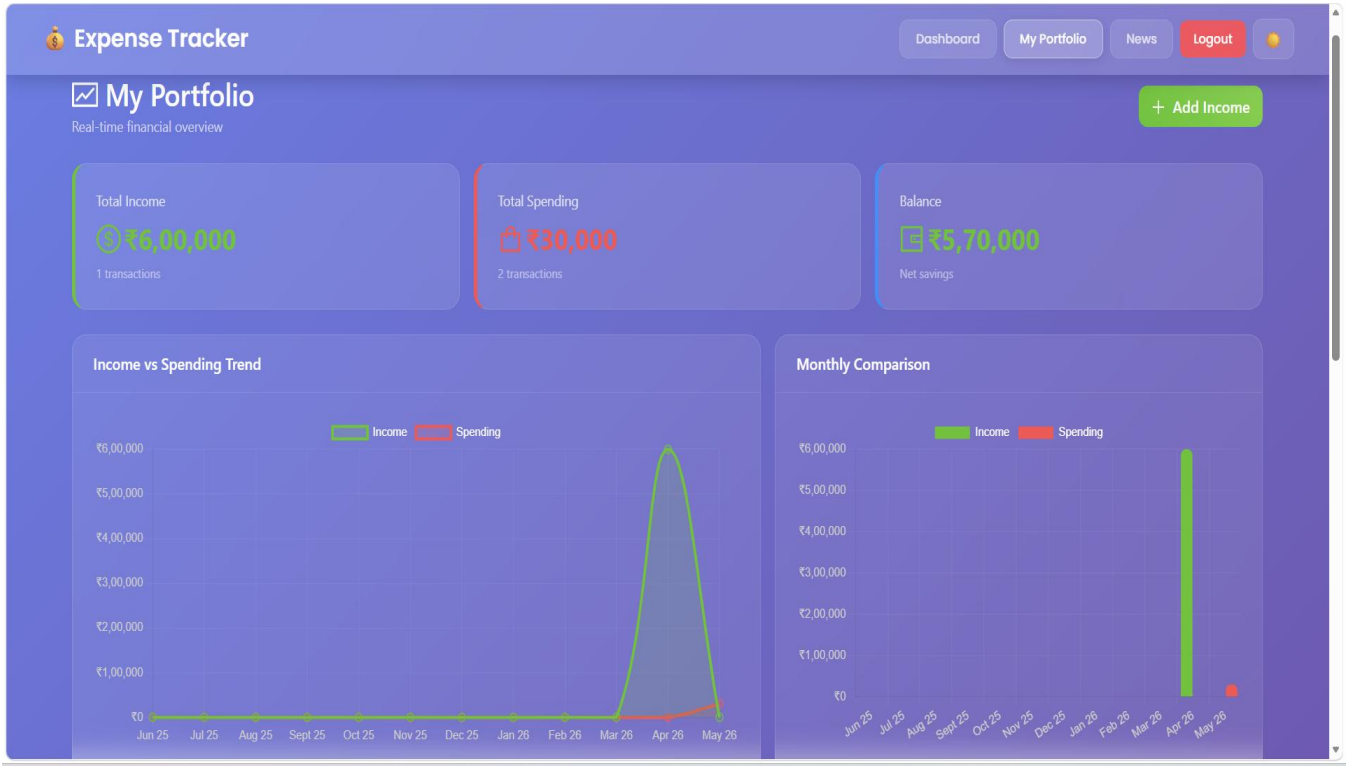
Download Expense Report

Download Expense Report

Last Month

Download PDF

Add Income



Recent Transactions

Recent Transactions

Type	Title	Amount	Category	Date	Action
Expense	bf expense	-₹20,000	jbsdxx	13/5/2026	
Expense	ujxbxbxe	-₹10,000	rxnebvcke	5/5/2026	
Income	600000	+₹6,00,000	Freelance	7/4/2026	

 Expense Tracker

DashboardMy PortfolioNewsLogout

 Financial News

Latest updates from Economic Times, LiveMint, MoneyControl & more

Refresh

DMart Q4 Results: Avenue Supermarts cons PAT jumps 19% to Rs 656 crore, revenue too rises 19%

DMart Q4 Results: Avenue Supermarts, the operator of D-Mart, on Saturday reported a net profit of Rs 656.6 crore for the March quarter of FY26, marking a jump of 19% from Rs 550.90...

Economic TimesMarkets44m ago

Read full article →

CDSL Q4 Results: Cons PAT declines 20% to Rs 80 crore, Rs 12.75/share dividend declared

CDSL Q4 Results: Central Depository Services Limited reported a mixed Q4 FY26 performance, with revenue growing 17.1% to Rs 262.85 crore, but net profit declining 20% to Rs 80.22 crore. Th...

Economic TimesMarkets2h ago

Read full article →

FII outflows not driven by lack of AI and high taxes, says Shankar Sharma. Here's why

Foreign investors are not avoiding India due to AI exposure or tax issues, according to investor Shankar Sharma. Data shows markets with little AI have outperformed. India's own past...

Economic TimesMarkets2h ago

Read full article →

Kotak Mahindra Bank Q4 Results: PAT jumps 13% to Rs 4,026.55 crore, NII rises 8%

Kotak Mahindra Bank Q4 Results: Kotak Mahindra Bank on Saturday reported a net profit of Rs 4,026.55 crore for the March quarter of FY26, marking a jump of 13.3% from Rs 3,55...

Economic TimesMarkets2h ago

Read full article →

IFSCA in talks with RBI over foreign asset reporting rule for GIFT City funds

The move follows an RBI FAQ that set out mandatory FLA filing requirements and raised questions over the regulatory classification of entities in the GIFT IFSC, prompting industry...

LiveMintMarkets2h ago

Read full article →

Nifty stays range bound as volatility rises; breakout awaited

Nifty 50 traded within a narrow range, closing slightly higher amid rising volatility and weak momentum indicators. Markets remain in consolidation, with key resistance and support levels...

Economic TimesMarkets3h ago

Read full article →

5.2 Functionality Evaluation

The Expense Tracker System was evaluated by testing all implemented features under real-time usage scenarios such as adding expenses, modifying records, and retrieving summaries.

Observed Results

- User registration and login functioned correctly
- Expense addition, update, and deletion worked without errors
- Category-based expense tracking was consistent
- Date-wise filtering provided accurate results
- Dashboard summary displayed correct total expenses
- Data was stored and retrieved reliably from MongoDB

Key Outcome

The system successfully handled core expense management operations.

All CRUD functionalities (Create, Read, Update, Delete) performed as expected, ensuring accurate tracking of user expenses with minimal delay.

5.3 User Experience Assessment

User experience was evaluated through basic usability testing with sample users interacting with the system.

Findings

- Interface was simple and easy to understand
 - Users could add and manage expenses without technical difficulty
 - Navigation between pages (login, dashboard, expense list) was smooth
 - Form inputs were minimal and easy to fill
 - Responsive design allowed usage on different screen sizes
-

User Feedback Highlights

- Users appreciated the simplicity of expense entry
- Quick access to expense history improved usability
- Dashboard overview helped in understanding spending patterns
- Reduced need for manual tracking (e.g., notebooks or spreadsheets)

5.4 Data Security and Integrity

Data security and integrity were evaluated to ensure safe handling of user information.

Security Measures Implemented

- Authentication using JWT-based login system
 - Password hashing for secure storage
 - Protected API routes using Express.js
 - Input validation to prevent invalid data
-

Integrity Results

- No data duplication observed during testing
 - Expense records remained consistent after multiple operations
 - Unauthorized access attempts were restricted
 - Data persisted correctly even after system restart
-

Overall Discussion

The results show that the Expense Tracker System effectively fulfills its primary objective of managing personal expenses in a simple and efficient way.

The system reduces manual effort involved in tracking expenses and provides a structured approach to financial management. While it does not include advanced features like multi-user synchronization or business accounting modules, it delivers reliable performance for individual users.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The **Expense Tracker System** was successfully designed and implemented as a web-based application to help users manage and track their daily expenses efficiently.

The system replaces traditional manual methods such as notebooks or spreadsheets by providing a structured digital platform for recording, updating, and analyzing expenses. By using the **MERN stack**, the application ensures smooth interaction between the user interface, backend logic, and database.

The core functionality of the system includes user authentication, expense management, and basic analytical features such as category-wise and date-wise tracking. These features allow users to monitor their spending habits and make better financial decisions.

Testing and evaluation show that the system performs reliably under normal usage conditions. All major functionalities such as adding, updating, deleting, and retrieving expense data work as expected. The system maintains data consistency and provides a simple and user-friendly interface for users with basic technical knowledge.

Overall, the Expense Tracker System achieves its primary objective of simplifying personal expense management. While it does not include advanced financial or multi-user accounting features, it provides a practical and efficient solution for individual users.

6.2 Future Scope

The Expense Tracker System can be further enhanced by incorporating advanced features and modern technologies to improve usability and functionality.

Possible Future Improvements

- **Mobile Application Development**
Extend the system into a dedicated Android/iOS app for better accessibility.
- **Advanced Data Visualization**
Add detailed charts and insights (monthly trends, spending patterns) using libraries like Chart.js.
- **Budget Planning Feature**
Allow users to set monthly budgets and receive alerts when limits are exceeded.
- **Cloud Synchronization**
Enable multi-device access with real-time data sync using cloud services.
- **Export Functionality**
Provide options to export reports in PDF and Excel formats.
- **Notification System**
Send reminders for daily expense entry or budget alerts.
- **Multi-language Support**
Make the application accessible to a wider audience.
- **Integration with Payment Systems**
Connect with UPI or banking APIs to auto-track expenses (advanced level).

6.3 Limitations

Despite its effectiveness, the current system has certain limitations:

Identified Limitations

- **Limited to Basic Expense Tracking**
Does not support advanced financial management or business accounting features.
- **No Real-Time Multi-User Functionality**
The system is designed for individual users only.
- **Basic Analytics**
Limited reporting and visualization compared to advanced financial tools.
- **Internet Dependency**
Requires an internet connection to access and store data.
- **Limited Security Features**
Uses basic authentication (JWT) without advanced security layers.
- **No External Integrations**
Currently not connected to banking systems or third-party services.

Chapter 7: References

7.1 REFERENCES

1. Roger S. Pressman & Bruce R. Maxim (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
2. Ian Sommerville (2016). *Software Engineering (10th Edition)*. Pearson Education.
3. Abraham Silberschatz, Henry F. Korth, & S. Sudarshan (2019). *Database System Concepts*. McGraw-Hill.
4. MongoDB Inc.. *MongoDB Documentation*. <https://www.mongodb.com/docs/>
5. Meta Platforms. *React Documentation*. <https://react.dev/>
6. OpenJS Foundation. *Node.js Documentation*. <https://nodejs.org/en/docs/>
7. Express.js Documentation. <https://expressjs.com/>
8. Postman Inc.. *Postman API Documentation*. <https://www.postman.com/>
9. IEEE (2014). *IEEE Standard for Software Requirements Specification*.
10. ISO. *ISO/IEC 25010:2011 – Software Quality Model*.
11. Google. *Material Design Guidelines*. <https://material.io/>
12. Various online tutorials and documentation related to MERN stack development, REST APIs, and web application design.